Technische Universität Braunschweig

Chair for Chip Design for Embedded Computing

HWSW-Systeme

VHDL-Übung 03: Einführung in VHDL

M.Sc. Eike Trumann, M.Sc. Jasper Homann

Prof. Dr.-Ing. Guillermo Payá Vayá

7. November 2025

Aufgabe VHDL-300: *Vier-1-Bit-Operand-Addierer*

VHDL-300

Für diese Aufgabe soll eine vereinfachte Variante des `best_calculator`, der bereits aus der Präsenzübung bekannt ist, mit einer Fuktionalität gefüllt werden. Die Funktionalität soll dabei ausschließlich aus den grundlegenden Logikelementen (AND, OR, XOR, NAND, NOR, ...) und der Verkettung (&) erstellt werden.

Es sollen vier parallel arbeitende 1-Bit-Addierer implementiert werden. Das bedeutet, es sollen je 2 Bit der Eingabe (beispielsweise `data_bus(0)` und `data_bus(1)`, die darauf folgenden auch jeweils paarweise) addiert werden und in der Ausgabe `result(0)` und `result(1)` ausgegeben werden. Das Ergebnis für die weiteren Bits der Eingabe wird dann in die folgenden Bits der Ausgabe geschrieben, beispielsweise kommt das Ergebnis von `data_bus(6)` + `data_bus(7)` in `result(6)` und `result(7)`. Dabei soll im geringwertigeren Bit (im Beispiel `result(0)` und `result(6)`) das Ergebnis der Addition und im höherwertigen Bit (im Beispiel `result(1)` und `result(7)`) der entstandene Übertrag gespeichert werden.

```
entity best_calculator is
port(
  data_bus      : in  std_ulogic_vector(7 downto 0);
  result        : out std_ulogic_vector(7 downto 0)
);
end best_calculator;
```

Aufgabe VHDL-301: *Schiebeoperand*

VHDL-301

Nun soll der `best_calculator` mit einer anderen Funktionalität gefüllt werden. Es handelt sich dabei um eine Schiebeoperation. Dabei soll der Eingabewert immer 2 Bit nach rechts verschoben werden, was einer Division durch 4 entspricht. Erschwerend kommt jedoch hinzu, dass zwei Arten von Schiebeoperationen, nämlich logische und arithmetische Schiebeoperationen unterstützt werden sollen. Um die durchzuführende Schiebeoperation auszuwählen, muss ein zusätzlicher Eingang hinzugefügt werden. Wenn der Eingang `logic_shift` eine 0 enthält, soll eine arithmetische Schiebeoperation (also unter Beachtung des Vorzeichens) durchgeführt werden. Enthält `logic_shift` hingegen eine 1, so soll eine logische Schiebeoperation ohne Beachtung des Vorzeichens erfolgen. Auch bei dieser Aufgabe sollen ausschließlich grundlegenden Logikelemente wie AND, OR oder XOR verwendet werden.

```
entity best_calculator is
port(
  data_bus     : in  std_ulogic_vector(7 downto 0);
  logic_shift  : in  std_ulogic;
  result       : out std_ulogic_vector(7 downto 0)
);
end best_calculator;
```

Aufgabe VHDL-302: *Testlogik (aus Klausur SoSe 2022)*

VHDL-302

Gegeben ist folgende *Entity* Definition eines Moduls:

```
entity tester is
        port (
                  A, B, C  : in  std_ulogic;
                  X, Y     : out std_ulogic);
end entity tester;
```

Beschreiben Sie die *Architecture* in VHDL, sodass das Modul die folgenden Binärausdrücke berechnet.

$$X = A \oplus (B \oplus C) \tag{1}$$

$$Y = AB\overline{C} \vee \overline{A}BC \vee A\overline{B}C \vee ABC \tag{2}$$

```
architecture rtl of tester is


begin




end architecture rtl;
```

Aufgabe VHDL-303: *Binärlogik (aus Klausur WiSe 2021)*

Gegeben ist folgende *Entity* Definition eines Moduls:

VHDL-303

```
entity tester is
  port (
    A,B,C,D  : in  std_ulogic;
    X, Y     : out std_ulogic);
end entity tester;
```

Beschreiben Sie die *Architecture* in VHDL, sodass das Modul die folgenden Binärausdrücke berechnet. Es soll möglichst wenig Logik gebaut werden.

$$X = (A \vee B) \wedge C \wedge D \tag{3}$$

$$Y = (C \wedge A) \vee A \vee B \tag{4}$$

```
architecture rtl of tester is


begin




end architecture rtl;
```